

Relatório Técnico - Trabalho 1



Alunos:

Bruno Jambeiro Mesquita RA: 260382

Lucas Rodrigues de Mendonça RA: 236800

Fernando Rodrigues da Silva RA: 247409

Thiago Augusto de Tulio Nascimento RA: 252937

Victor Itiro Ogitsu RA: 244075

Campinas

Abril 2026

Os arquivos podem ser encontrados no [repositório do projeto](#).

Sumário

1	Modelagem do problema	2
2	Algoritmos de Busca	2
2.1	Implementação do Minimax	2
2.2	Implementação do Alfa-Beta	3
2.3	Estratégia de Ordenação de Jogadas	3
2.4	Iterative Deepening	3
2.5	Análise do Impacto da Poda e Complexidade	3
3	Funções Heurísticas	3
3.1	Heurística Gulosa (Greedy)	3
3.2	Heurística Posicional Estática (Static)	3
3.3	Heurística Dinâmica (Dynamic)	4
4	Avaliação experimental	4
4.1	Comparação entre Minimax e Alpha-Beta;	4
4.2	Comparação entre heurísticas;	4
4.3	Profundidade média atingida;	4
4.4	Número médio de nós expandidos;	5
4.5	Tempo médio por jogada;	5
4.6	Taxa de vitória contra baseline.	5
5	Discussão	5
5.1	Limitações do agente	5
5.2	Gargalos computacionais	5
5.3	Relação entre profundidade e qualidade de jogo	6
5.4	Complexidade prática observada	6

1 Modelagem do problema

Seja $P = \{\mathbf{b}, \mathbf{w}\}$ o conjunto dos jogadores, que controlam as peças pretas e brancas, respectivamente. Seja $C = \{(x, y) \in \mathbb{N}^2 \mid 0 \leq x \leq 7 \wedge 0 \leq y \leq 7\}$ o conjunto de coordenadas do tabuleiro 8x8. Seja $V = P \cup \{\mathbf{v}\}$ o conjunto que determina os possíveis valores de cada coordenada no tabuleiro.

Uma disposição das peças no tabuleiro é dada por uma função $B : C \rightarrow V$. Dado $c \in C$:

$$B(c) = \begin{cases} \mathbf{b}, & \text{caso a peça em } c \text{ seja preta} \\ \mathbf{w}, & \text{caso a peça em } c \text{ seja branca} \\ \mathbf{v}, & \text{caso não haja peça em } c \end{cases}$$

Definimos um **estado** s do jogo como a tupla (B, p) , onde B representa a disposição atual de peças no tabuleiro e $p \in P$ indica de qual cor é a próxima jogada. Também definimos um **conjunto de estados** S , tal que $S = \{(B, p) \mid B \text{ é um tabuleiro válido} \wedge p \in P\}$. Consideramos válido um tabuleiro que pode ser alcançado a partir de jogadas válidas sobre o estado inicial e seus sucessores.

Dado um estado s , a **função de ações** $A(s)$ retorna o conjunto de movimentos legais para o jogador p . Seja $D = \{-1, 0, 1\}^2 \setminus \{(0, 0)\}$ o conjunto de direções válidas para flanqueamento.

Um movimento na coordenada c é válido para o jogador $p \iff [B(c) = \mathbf{v} \wedge \exists (d, k) \in D \times (\mathbb{N}^* \setminus \{1\}) \text{ tal que } (B(c + kd) = p \wedge \forall 0 < i < k \{B(c + id) = p'\})]$, onde p' é a cor do oponente.

Seja $M(s) \subset C$ o conjunto de todas as coordenadas c que satisfazem a condição de flanqueamento acima. Definimos a função de ações como:

$$A(s) = \begin{cases} M(s), & \text{se } M(s) \neq \emptyset \\ \{\mathbf{passar}\}, & \text{se } M(s) = \emptyset \end{cases}$$

Seja $F(a)$ o conjunto de todas as coordenadas cujas peças são flanqueadas (viradas) quando a ação a é executada. Utilizando a mesma lógica de direção d e distância k , definimos esse conjunto como:

$$F(a) = \{c \in C \mid \exists (d, k) \in D \times (\mathbb{N}^* \setminus \{1\}) \text{ tais que } B(a + kd) = p \wedge (\forall 0 < j < k, B(a + jd) = p') \wedge (\exists 0 < i < k \text{ tal que } c = a + id)\}$$

Dados um estado $s = (B, p)$ e uma ação $a \in A(s)$, a **função de transição** $T : S \times A \rightarrow S$ retorna qual será o novo estado do jogo caso o jogador que detém o turno (p) tome a ação a no estado s :

$$T(s, a) = \begin{cases} (B, p'), & \text{se } a = \mathbf{passar} \\ (B', p'), & \text{caso contrário} \end{cases} \quad \text{De forma que:} \quad B'(c) = \begin{cases} p, & \text{se } c = a \\ p, & \text{se } c \in F(a) \\ B(c), & \text{caso contrário} \end{cases}$$

O conjunto de estados terminais $S_{terminal} \subset S$ é alcançado quando nenhum dos jogadores possui movimentos de flanqueamento válidos. Ou seja, um estado $s = (B, p)$ é terminal se $A((B, p)) = \{\mathbf{passar}\}$ e $A((B, p')) = \{\mathbf{passar}\}$.

Para qualquer estado $s \in S_{terminal}$, definimos $|B_p|$ como a cardinalidade do conjunto de células ocupadas pelo jogador p (ou seja, $|\{c \in C \mid B(c) = p\}|$). A função de utilidade $U : S_{terminal} \times P \rightarrow \mathbb{Z}$ avalia o resultado do ponto de vista do jogador p como o saldo exato de material:

$$U(s, p) = |B_p| - |B_{p'}|$$

2 Algoritmos de Busca

2.1 Implementação do Minimax

O algoritmo Minimax é a base da tomada de decisão em jogos de soma zero bidimensionais. A implementação desenvolvida baseia-se na exploração de uma árvore de estados onde os nós representam as possíveis configurações do tabuleiro. O agente (maximizador) busca maximizar sua função heurística de ganho, enquanto assume que o oponente (minimizador) jogará de forma otimizada para minimizar esse mesmo valor. Devido à explosão combinatoria do Othello, a busca é limitada a uma profundidade máxima obtida dentro de um tempo pré-definido de 0.95s, momento em que a função de avaliação estática é chamada para estimar a utilidade do estado folha.

2.2 Implementação do Alfa-Beta

Para otimizar o Minimax padrão, foi integrada a poda Alfa-Beta ($\alpha - \beta$), que reduz drasticamente o número de nós avaliados sem alterar a decisão final do algoritmo. A técnica mantém duas variáveis limitantes durante a travessia em profundidade: α , que representa o valor máximo garantido para o maximizador, e β , o valor mínimo garantido para o minimizador. Durante a expansão de um nó, se o algoritmo detectar que $\beta \leq \alpha$, o ramo atual é podado. Isso ocorre pois já se sabe que o oponente possui uma refutação prévia que impede que aquele estado subótimo seja alcançado em uma partida real.

2.3 Estratégia de Ordenação de Jogadas

A eficiência da Poda Alfa-Beta é altamente dependente da ordem em que os nós são visitados. Para maximizar os cortes na árvore de busca, foi implementada uma estratégia de ordenação de jogadas. Antes de expandir os filhos de um nó, os movimentos válidos são pré-avaliados e ordenados. O maximizador explora primeiro as jogadas que geram os maiores valores heurísticos imediatos (capturas de cantos e bordas), enquanto o minimizador explora primeiro as de menores valores. Esta heurística gulosa aumenta a probabilidade de encontrar um valor ótimo cedo, estreitando a janela $[\alpha, \beta]$ e forçando podas massivas nos estágios iniciais da busca.

2.4 Iterative Deepening

Para contornar o problema de limites de tempo estritos por turno, o agente utiliza o Iterative Deepening com limite de tempo. Em vez de iniciar uma busca estática diretamente na profundidade d , o algoritmo executa o Minimax repetidamente para profundidades crescentes $(1, 2, 3, \dots, d)$, até que o tempo limite da jogada se esgote. Além de garantir que o agente sempre retorne um movimento válido caso seja interrompido, o Iterative Deepening atua junto com a ordenação de jogadas: o "melhor movimento" da iteração $d - 1$ é salvo em memória e utilizado como o primeiro ramo a ser explorado na iteração d .

2.5 Análise do Impacto da Poda e Complexidade

A introdução do Alfa-Beta aliada à ordenação de jogadas altera substancialmente a complexidade de tempo do agente. Enquanto o Minimax puro possui complexidade temporal estrita de $O(b^d)$, onde b é o fator de ramificação médio e d é a profundidade da busca, a poda bem-sucedida mitiga esse custo. No melhor caso teórico, onde a ordenação de jogadas é perfeita, a complexidade cai para $O(b^{d/2})$, permitindo ao agente explorar o dobro de profundidade no mesmo intervalo de tempo. Na prática, a implementação desenvolvida atinge uma redução significativa do espaço de estados processado, viabilizando buscas responsivas e competitivas sob restrições de tempo real.

3 Funções Heurísticas

Para orientar a tomada de decisão do agente e guiar as podas na árvore de busca, foram projetadas e implementadas três funções heurísticas de complexidades crescentes. Estas funções são submetidas a uma avaliação experimental comparativa para determinar a abordagem empírica mais eficiente.

3.1 Heurística Gulosa (Greedy)

A primeira heurística atua como o baseline do projeto. Simplesmente calcula de peças (diferença entre o número de discos do maximizador e do minimizador). Assim, o agente prioriza caminhos na árvore de busca que resultem no maior acúmulo numérico de peças a seu favor no estado avaliado.

3.2 Heurística Posicional Estática (Static)

A segunda abordagem introduz o conceito de controle de território. Ela estabelece uma matriz de pesos 8×8 , onde cada coordenada do tabuleiro recebe uma pontuação pré-definida. Posições estrategicamente superiores e inconvertíveis, como os cantos, recebem valores intensamente positivos. Em contrapartida, as casas adjacentes a eles — que concedem os cantos ao adversário — recebem penalidades. O valor de utilidade do estado é obtido pelo produto escalar entre as posições ocupadas e a matriz de pesos.

3.3 Heurística Dinâmica (Dynamic)

Essa heurística é a mais sofisticada, reconhecendo que o valor das estratégias no Othello sofre mutações conforme a fase da partida (mensurada pela quantidade de casas vazias). Trata-se de uma combinação linear que unifica o saldo, o mapa posicional estático e uma nova métrica de Mobilidade. Os três fatores são calculados em todos os turnos, mas seus pesos variam dinamicamente conforme o momento do jogo:

- **Início de Jogo (Early Game):** O maior peso multiplicador é atribuído à mobilidade, buscando maximizar as opções legais de jogada enquanto tenta restringir o adversário.
- **Meio de Jogo (Mid Game):** O peso da avaliação posicional (Static) torna-se predominante, forçando o agente a priorizar posições seguras e a captura de bordas e cantos.
- **Fim de Jogo (Late Game):** Quando restam poucos espaços livres e a árvore de busca aproxima-se do fim real da partida, a métrica posicional é suprimida e o peso material (Greedy) passa a dominar a equação de forma absoluta.

Esperava-se que o modelo Dynamic apresentasse a maior taxa de sucesso devido à sua capacidade de adaptação tática e maior complexidade estratégica. Entretanto, o comportamento do agente revelou fenômenos contraintuitivos decorrentes das restrições de tempo computacional.

4 Avaliação experimental

Para validar o desempenho e a eficiência foram realizados torneios automatizados. Cada confronto alternou a cor inicial dos agentes para mitigar a vantagem do primeiro jogador e utilizou aberturas aleatórias para garantir diversidade de estados. Todos os testes foram conduzidos sob o mesmo orçamento computacional, com um limite estrito de 0.95s por jogada.

4.1 Comparação entre Minimax e Alpha-Beta;

Para avaliar o impacto da poda, realizaram-se partidas espelho (agentes utilizando a mesma heurística), onde um lado possuía a poda Alpha-Beta desativada. Como esperado, a versão otimizada obteve superioridade. No confronto da heurística *Static*, o agente com poda venceu 75.0% das partidas. O que é explicado pela capacidade exploratória: Enquanto o Minimax puro alcançou uma profundidade média de 9.00 camadas, o agente com Alpha-Beta atingiu a profundidade de 12.23 no mesmo intervalo de tempo de 0.86s a 0.88s. Isso comprova que a poda permite explorar cenários muito mais profundos e relevantes sem estourar o orçamento de tempo

4.2 Comparação entre heurísticas;

Inicialmente, conjecturou-se que a heurística *Dynamic*, por ser a mais complexa e contemplar transições de estratégia ao longo do jogo, obteria o melhor desempenho. E embora a mais simples *Greedy* de fato tenha tido o pior desempenho, ao testar os modelos com diferentes permissões de profundidade máxima, os testes revelaram que o algoritmo *Static* de avaliação posicional obteve a maior taxa de vitória contra todos os seus adversários. No confronto direto, a heurística *Static* derrotou a *Dynamic* em 79.2% das partidas.

4.3 Profundidade média atingida;

A profundidade média obtida variou dependendo da heurística escolhida: Devido à quantidade de tempo limitado por jogada, as heurísticas mais simples e que portanto realizam menos cálculos conseguiram alcançar uma profundidade maior, enquanto a mais complexa foi mais limitada. Isso pode ser um dos motivos que resultou no melhor desempenho do modelo posicional, que conseguiu considerar muito mais cenários do que o modelo dinâmico. O momento do jogo também influencia bastante. O mid-game apresenta muitas possibilidades de movimento a serem realizados, reduzindo bastante o a profundidade atingida. O late game se torna mais travado, atingindo um valor extremamente alto(30+), que distorce a média.

Greedy: Profundidade mediana 5, Profundidade média 11.18 (Vitória do humano) Static: Profundidade mediana 5, Profundidade média 14.9 Dynamic: Profundidade mediana 5, Profundidade média 13.3

4.4 Número médio de nós expandidos;

O valor varia bastante de acordo com o momento do jogo, mas durante os experimentos obtivemos uma média de: 4.87.

4.5 Tempo médio por jogada;

O tempo por jogado é essencialmente o tempo máximo do iterative deepening, então 0.95 segundos. No fim do jogo ele consegue terminar mais rápido.

4.6 Taxa de vitória contra baseline.

Considerando a heurística *Greedy* como o baseline do projeto, os algoritmos avançados obtiveram excelentes taxas de vitória. O modelo *Dynamic* superou o baseline com 79.2% de vitórias, enquanto o modelo *Static* praticamente o aniquilou, com 95.8%. Os dados demonstram que, no Othello, a captura imediata de peças é uma tática falha frente ao controle territorial. Ademais, prova-se que a adição de camadas de complexidade estratégica (como no modelo *Dynamic*) só é vantajosa se não atrasar o processador a ponto de causar "miopia temporal" na árvore de busca.

5 Discussão

5.1 Limitações do agente

A principal limitação do agente está na explosão exponencial do espaço de estados à medida que a profundidade da busca aumenta. De acordo com os experimentos realizados pelo grupo, cada estado permite, em média, cerca de 7 movimentos possíveis. Como consequência, para analisar todas as sequências de até 12 jogadas à frente, seria necessário explorar aproximadamente 5 trilhões de estados. Técnicas de otimização, como o alpha-beta pruning, ajudam a reduzir esse número na prática, mas não eliminam a natureza exponencial do problema. Diante desse crescimento explosivo do espaço de possibilidades, torna-se inviável garantir a escolha da jogada globalmente ótima no jogo de Othello utilizando essa abordagem. Em vez disso, o agente é capaz de determinar a melhor jogada apenas dentro de um horizonte limitado de busca, maximizando uma função heurística. Assim, cabe ao programador projetar uma heurística que reflita adequadamente os objetivos do jogo, orientando o agente a tomar decisões que aumentem suas chances de vitória. Isso evidencia outra limitação importante do modelo: sua incapacidade de aprender e atualizar automaticamente a própria heurística. Diferentemente de abordagens baseadas em aprendizado por reforço, como o AlphaZero, desenvolvido pela Google (por meio da DeepMind), o modelo apresentado neste trabalho depende fortemente de heurísticas previamente definidas para alcançar bom desempenho. Como consequência, sua eficácia está diretamente condicionada ao conhecimento de jogo incorporado por seus criadores, o que pode limitar sua capacidade de adaptação a diferentes estratégias ou cenários não previstos. Além disso, por não aprender com a experiência, o agente não melhora ao longo do tempo, mantendo sempre o mesmo padrão de decisão, independentemente dos resultados obtidos em partidas anteriores.

5.2 Gargalos computacionais

Para Verificar os Gargalos computacionais foi utilizado cProfile em conjunto com o snakeviz para visualização fácil.

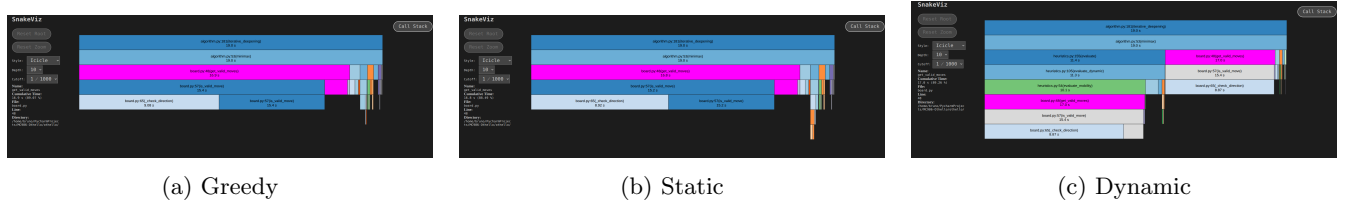


Figure 1: Gráficos de performance do Minimax

A heurística static e greedy tem perfis bem parecidos, mas a heurística dynamic se mostra muito mais pesada computacionalmente. Tal diferença computacional pode explicar a sua pior performance comparada com a static, apesar de realizar uma análise mais complexa.

Em ambos os casos, o gargalo computacional é na geração e verificação de movimentos novos.

5.3 Relação entre profundidade e qualidade de jogo

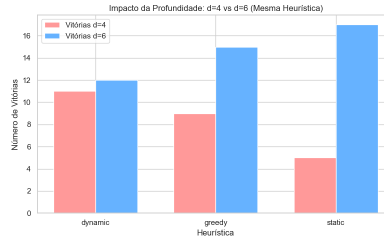
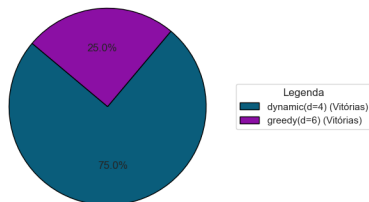


Figure 2: Gráficos de Vitórias de Jogos com mesma heurística

Os resultados indicam uma relação clara e consistente entre o aumento da profundidade máxima no iterative deepening e a taxa de vitória, especialmente quando se comparam agentes do mesmo tipo. Nos confrontos homogêneos, como dynamic(d=4) vs dynamic(d=6), greedy(d=4) vs greedy(d=6) e static(d=4) vs static(d=6), o agente com maior profundidade venceu cerca de 52% a 80% das partidas. Isso evidencia que, mantendo fixa a heurística, explorar mais profundamente a árvore de jogo aumenta significativamente a qualidade das decisões, reduzindo erros táticos e melhorando a avaliação de consequências futuras.

Todavia, profundidade não parece compensar pela qualidade de heurística, com as heurísticas mais avançadas, como dynamic e o static, ganhando consistentemente da heurística greedy, mesmo com diferença de profundidade

Confronto: dynamic(d=4) vs greedy(d=6)



Confronto: static(d=4) vs greedy(d=6)

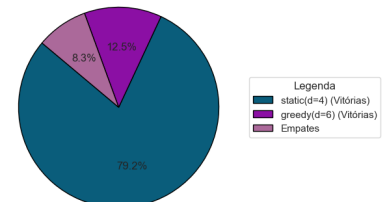


Figure 3: Gráficos de Vitórias de Jogos com heurística e profundidades diferentes

5.4 Complexidade prática observada

Na prática, a complexidade observada do agente manifesta-se principalmente como o nível de esforço computacional necessário para tomar uma decisão dentro do tempo disponível, e não apenas como crescimento assintótico. Os experimentos mostram que esse custo varia significativamente ao longo da partida e entre diferentes heurísticas: no meio de jogo, a grande quantidade de movimentos possíveis torna a decisão mais “difícil”, exigindo mais processamento para avaliar alternativas relevantes, enquanto no final do jogo a redução de opções torna as decisões mais rápidas e profundas. Além disso, heurísticas mais elaboradas aumentam o custo por avaliação, o que reduz a quantidade de cenários que podem ser analisados no mesmo intervalo de tempo. Assim, a complexidade prática do agente está diretamente ligada a esse equilíbrio entre quantidade de posições exploradas e custo de avaliar cada uma delas. Em termos concretos, decisões mais “complexas” surgem quando há muitas alternativas plausíveis e pouco tempo para diferenciá-las, evidenciando que a dificuldade real do problema está menos na fórmula teórica e mais na gestão eficiente de recursos computacionais frente à dinâmica do jogo.